

## Article

# Multi-Agent-Based Smart-Home Energy Management with Adaptive Reasoning

Elena Dolinin <sup>1,\*</sup> and Chairi Kiourt <sup>1,2</sup>

<sup>1</sup> MSc Cognitive Systems, Faculty of Pure and Applied Sciences, Open University of Cyprus, 2252 Nicosia, Cyprus

<sup>2</sup> Athena-Research Centre, 67100 Xanthi, Greece, chairiq@athenarc.gr

\* Correspondence: elena.dolinin@st.ouc.ac.cy

## Abstract

This paper introduces SmartHouseOperator, a multi-agent intelligent control framework for adaptive and energy-efficient smart-home management. Modern smart homes integrate heterogeneous devices and sensors, yet most existing solutions rely on static rules or manual coordination, limiting their ability to adapt to dynamic environmental conditions and evolving user preferences. SmartHouseOperator addresses these limitations through an agentic architecture that coordinates device-specific agents for air conditioning, lighting, refrigeration, and shutters under a central orchestrator. The system combines contextual inputs (e.g., weather, occupancy, power load), persistent knowledge, reinforcement-learning-based preference modeling, and LLM-powered reasoning to enable coordinated and personalized control decisions. Experimental results show that the framework achieves consistent reasoning performance across multiple agent orchestration engines and reduces air-conditioning power consumption by up to 16% under critical load conditions. These findings demonstrate the potential of multi-agent, learning-enabled control systems to deliver intelligent, energy-aware, and user-centric smart-home operation.

**Keywords:** smart homes; multi-agent systems; agentic AI; reinforcement learning; energy-aware control; energy efficiency; IoT

## 1. Introduction

Modern smart homes are becoming increasingly complex environments, integrating a growing number of heterogeneous devices, sensors, and data sources. While these systems promise convenience, safety, and energy efficiency, in practice they often fall short of their full potential. Smart homes integrate diverse devices and communication technologies that introduce system complexity and interoperability challenges [1]. Most current energy management and automation schemes rely on manual coordination or static control strategies that do not adapt to dynamic environmental conditions or user behavior, leading to inefficiencies and inconsistent comfort [2]. As the number of connected devices increases, so does the difficulty of managing them—in part because heterogeneous IoT devices often cannot coordinate effectively and may exhibit conflicting behaviors [3].

A major challenge arises from the dynamic nature of real homes. Environmental conditions such as weather, daylight and indoor temperature fluctuate throughout the day. User behavior also changes—people come and go, their comfort preferences vary, and

Academic Editors: Hyokyung Bahn and Douglas O'Shaughnessy

Received: 30 December 2025

Revised: 6 February 2026

Accepted: 11 February 2026

Published: 13 February 2026

**Copyright:** © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

their routines shift. Contemporary smart-home systems rarely adapt to these real-time dynamics [4]. Instead, they follow fixed rules, static schedules, or vendor-provided automation presets that cannot generalize across contexts. This often results in energy waste, overcooling/heating, suboptimal lighting, or conflicts between devices. Moreover, smart homes lack the ability to reason holistically: each device acts independently, without understanding how its decisions affect overall energy usage. At the same time, personal preferences—such as temperature comfort or lighting brightness, are not modeled in a principled way and are almost never learned from experience [5]. This prevents smart homes from delivering personalized efficiency.

Despite rapid advances in smart-home technologies, most deployed systems remain device-centric and reactive, relying on static rules or predefined schedules. As a result, they struggle to adapt to changing environmental conditions, evolving user behavior, and system-level energy optimization requirements. Existing solutions typically address devices in isolation, offering limited coordination, personalization, or holistic reasoning across heterogeneous components. This gap motivates the exploration of intelligent control frameworks that can operate at the system-level and adapt continuously to real-world household dynamics.

To address these limitations, there is a growing need for autonomous, adaptive, decision-making systems capable of reasoning across devices and optimizing their behavior in a coordinated manner. Integrating agentic artificial intelligence (AI) and reinforcement learning (RL) offers a pathway to such intelligent control. Agent-based architectures enable devices to act as rational, reactive and pro-active decision-makers that can sense their environment, communicate with each other, and plan actions [6,7]. RL allows the system to learn user preferences and adjust behavior over time, while multi-agent collaboration ensures consistent global policies.

This paper introduces the SmartHouseOperator, a multi-agent intelligent control framework designed to optimize household energy consumption while maintaining safety and user comfort. The system integrates specialized agents for heterogeneous smart-home devices—such as air conditioning, lighting, refrigerators, and shutters, coordinated through a Manager Agent capable of reasoning over global constraints. It leverages contextual inputs, including weather conditions, power consumption levels, occupancy patterns, and learned user preferences, together with reinforcement-based personalization and reasoning mechanisms powered by large language models (LLMs). In addition, integrated knowledge and memory modules allow general or device-specific rules to be provided to agents or retrieved from past experience to support informed decision-making. Through a closed-loop pipeline, the system continuously re-evaluates environmental conditions and autonomously applies actions that minimize energy usage without compromising comfort. Moreover, by combining agentic AI, real-time contextual reasoning, and RL-driven personalization, the SmartHouseOperator demonstrates how next-generation smart homes can evolve into adaptive, efficient and intelligent environments capable of responding to dynamic contexts and reducing energy waste [8,9].

The scientific methodology adopted in this study follows a simulation-driven evaluation framework designed to ensure controlled, repeatable, and realistic assessment of adaptive smart-home control. The simulation model was developed to represent a typical residential environment and includes heterogeneous high-impact devices (air conditioning, lighting, shutters, and refrigeration), selected based on their relevance to household energy consumption and comfort. Contextual inputs such as weather conditions, time of day, occupancy, and power-load level were incorporated to reflect real-world operating conditions.

System parameters, including device power baselines, operational limits, and learning rates, were defined using manufacturer specifications and values commonly reported

in the smart-home energy literature. Boundary conditions related to energy consumption were explicitly enforced through discrete power-load modes (Normal, Moderate, Critical), which constrain allowable device actions and prevent infeasible or unsafe behavior. Result analysis combines qualitative evaluation of multi-agent reasoning—based on pre-defined criteria for personalization accuracy, contextual responsiveness, interpretability, and response time—with quantitative estimation of energy savings, focusing on air-conditioning power consumption under different load scenarios.

## 2. Materials and Methods

This study is supported by an open-source implementation of the SmartHouseOperator framework, available at <https://github.com/edolinina/smart-house-operator> (accessed on 08 Dec 2025), which contains all source code, configurations, simulated datasets, and evaluation scripts required to reproduce the reported results. The experiments were conducted in a real smart-home environment using simulated scenarios and logged contextual inputs; no human or animal subjects were involved, and therefore no ethical approval was required. Generative AI tools were used in a limited and disclosed manner: the system architecture figure was created with assistance from an AI-based diagramming tool (ClickUp AI, version 4.0, <https://app.clickup.com>, accessed on 08 Dec 2025) based on a human-defined conceptual design, and Microsoft Word Copilot was used solely for language polishing and grammar refinement. No AI system was used to generate technical content, experimental data, analysis, or conclusions.

## 3. Background—Brief Literature Review

Recent advances in intelligent home automation increasingly rely on agentic AI systems, which combine reasoning capabilities of LLMs with grounded interactions in physical or simulated environments [10,11]. These systems aim to move beyond static, rule-based smart-home automation toward adaptive, context-aware agents that can interpret user instructions, coordinate multiple devices, and perform multi-step tasks autonomously. A central challenge in this domain is bridging the gap between high-level language understanding and reliable low-level device control—particularly when devices expose heterogeneous application programming interfaces (APIs), lack semantic descriptions, or provide limited feedback. Several frameworks have emerged to address this gap by integrating LLM planning, tool invocation, and environment grounding [12].

One influential example of this line of work is the SAGE (Smart-home Agent with Grounded Execution) framework, which demonstrates how structured LLM-driven control can enable robust execution of complex home-automation tasks [13]. Rivkin et al. present SAGE as a framework that leverages LLMs as high-level planners for interpreting natural user commands and executing multi-step smart-home tasks. Unlike rule-based approaches, SAGE grounds LLM reasoning through a structured execution loop that includes API-schema reading, user preference management and device-state manipulation and monitoring. Key components of the system include: a personalization tool that learns user preferences from past interactions; a device-interaction tool capable of controlling any smart device using its API documentation; a device-disambiguation tool that resolves ambiguous device references using a single home photo; and persistent command execution through LLM-generated monitoring routines. These tools are integrated into a unified framework with a single entry point, specifically designed to mitigate common LLM limitations in smart-home environments. The study evaluates SAGE on a benchmark of 50 complex household tasks and demonstrates higher reliability, flexibility, and task performance compared with existing LLM-based automation approaches, including unguided LLM prompting. Compared to SAGE, the proposed approach differs fundamentally in

both architectural design and reasoning mechanisms. While SAGE employs a single LLM-centered planning loop augmented with long-term memory and user profiling, the proposed method adopts a multi-agent architecture, where specialized agents operate concurrently and coordinate through structured interfaces. This design enables explicit separation of concerns between perception, decision-making, and actuation, improving modularity and extensibility. In terms of reasoning, SAGE infers user preferences deterministically through LLM-based memory and prompt-driven personalization. In contrast, the proposed system incorporates lightweight reinforcement-learning components to model user preferences as probabilistic distributions over actions, allowing adaptation under uncertainty and gradual preference refinement rather than single-point estimates. This enables the system to represent multiple plausible user outcomes and adjust behavior incrementally based on observed feedback. From a deployment perspective, SAGE relies on commercial, cloud-hosted LLMs and extensive prompt engineering with Retrieval-Augmented Generation (RAG) over API documentation, limiting execution in offline, proprietary, or resource-constrained environments. The proposed framework is designed to operate with local or self-hosted models, minimal prompting, and configuration-driven agent behavior, enabling broader applicability across heterogeneous and less standardized devices beyond the SmartThings ecosystem.

Overall, the proposed method emphasizes distributed agentic control, probabilistic personalization, and deployment flexibility, rather than centralized LLM planning, distinguishing it from SAGE at both the architectural and reasoning levels.

Latoń, Grela & Ożadowicz [14] provide a comprehensive review of how Deep Reinforcement Learning (DRL) methods are being used in Home Energy Management Systems (HEMS) to optimize residential energy consumption. They demonstrate that DRL—by exploiting neural-network as RL function approximation, is especially well suited to the complex, dynamic, and real-time nature of modern homes, characterized by many smart devices, variable demand, renewable integration, and user comfort constraints. The paper shows that while DRL-based HEMS perform well in simulations, significant gaps remain before they can be effectively deployed in real-world settings. A major challenge is the lack of empirical studies, as most work relies on digital twins that require extensive, high-quality, high-frequency datasets—such as device-level consumption, weather, and occupancy patterns, to accurately model physical environments; without such data, RL agents risk learning unrealistic policies. Additional issues include difficulties achieving scalability and interoperability across heterogeneous devices, and strong dependence of RL decision-making on data completeness and accuracy, alongside heightened privacy and security concerns. Overall, the paper argues that while DRL offers powerful tools to make smart homes more efficient and autonomous, there remains a significant gap between academic models and practical, scalable implementations.

Jimenez-Romero et al. [15] in his work explored another innovative framework that integrates LLMs into Multi-Agent Systems (MAS), replacing traditional, manually coded agent logic with prompt-driven behaviors. The authors demonstrate this approach using classic multi-agent models from swarm intelligence—such as ant-colony foraging and bird flocking—implemented in a simulation environment. In their system, each agent's decisions are guided by an LLM, which receives the agent's current local observations and returns actions via structured prompts. The agents may operate under purely LLM-driven behavior or as hybrids mixed with rule-based agents, allowing the study of emergent collective behavior under different prompting strategies. The results show that LLM-powered agents can achieve performance comparable to classical rule-based agents, and that hybrid populations often produce efficient or even improved collective behaviors. This suggests that LLM-driven agents are viable for complex, distributed, and adaptive multi-agent systems, offering flexibility and reducing the need for domain-specific rule

engineering. However, unlike SmartHouseOperator architecture, this work does not address device-level control, real-world actuators, contextual grounding in a physical environment, or user-specific personalization. It operates in purely simulated environments, with abstract agents and idealized observations, rather than interfacing with diverse real smart-home devices. Thus, while this work validates the concept of LLM-driven MAS in simulation, it is of limited applicability to real-world scenarios, structured device interaction, and user-centric adaptive control.

The paper of Giudici et al. [16] explores the use of LLMs to ease smart-home automation by allowing users to create automation routines (trigger-action rules) in natural language, rather than writing code manually. By integrating LLMs with the HomeAssistant framework [17], the system translates user requests into valid automation configurations (JavaScript Object Notation (JSON)) that HomeAssistant can execute. This approach lowers the barrier for non-technical users, enabling them to specify complex automation behaviors via simple conversational prompts. In empirical evaluation with 56 users, the LLM-based chatbot generated routines that users found intuitive and useful; although some output suffered from syntax errors or malformations, the overall user satisfaction and engagement were high. The study demonstrates that LLM-powered conversation agents can significantly improve accessibility and flexibility in smart-home automation, supporting sustainable and user-friendly energy management. However, the system remains focused on generating one-off automation rules rather than supporting continuous, autonomous, context-aware and personalized house control.

Unlike previous approaches, the SmartHouseOperator emphasizes lightweight, configurable, and extensible control [7]. It enables agents to collaborate and operate without heavy prompts, using YAML (YAML Ain't Markup Language)-based configurations to define rules, embedded knowledge, or agent-specific behaviors [6]. The system supports adaptive personalization by leveraging real-time contextual inputs and learned user preferences [18], while its knowledge and memory integration enables retrieval of past experience and application of general or device-specific rules [19]. Moreover, the framework is designed to be modular, allowing the use of different LLMs, agentic orchestration frameworks, and additional devices or inputs, thereby facilitating scalable and user-centric smart-home automation.

Related approaches in the literature have proposed learning- and agent-based control strategies for automated decision-making and coordination under uncertainty, often grounded in control-theoretic formulations or optimization-driven architectures [20–22]. While these methods provide strong guarantees in structured or industrial settings, they typically assume tightly coupled system dynamics, abstract state representations, or centralized control assumptions. In contrast, SmartHouseOperator prioritizes practical deployment in heterogeneous, user-facing environments by combining LLM-driven agentic reasoning with lightweight preference learning and configuration-based control, trading formal optimality for flexibility, interpretability, and ease of integration with real smart-home devices.

Table 1 provides a compact qualitative comparison of representative smart-home AI approaches discussed above, highlighting key architectural characteristics and deployment capabilities. The reviewed approaches, including the proposed framework, make several methodological assumptions and face practical deployment constraints. In particular, LLM-driven control assumes the availability of reliable contextual inputs, sufficiently expressive device interfaces, and stable interpretation of natural language instructions. In real-world smart-home environments, these assumptions may be challenged by heterogeneous device APIs, incomplete feedback, latency, and safety considerations. Furthermore, LLM-based agents may produce inconsistent or non-deterministic outputs, require careful prompt design, and exhibit limited robustness when operating under



resource constraints or without external validation mechanisms. These limitations motivate the use of structured control loops, lightweight learning components, and explicit validation layers to improve robustness in practical deployments.

**Table 1.** High-level qualitative comparison of representative smart-home AI approaches.

| Approach                              | LLM-Based Reasoning | Multi-Agent | Personalization  | Real-World Deployment |
|---------------------------------------|---------------------|-------------|------------------|-----------------------|
| SAGE [13]                             | ✓                   | ✗           | ✓ (LLM-inferred) | ✓                     |
| DRL-based HEMS [15]                   | ✗                   | ✗           | ✓ (policy-based) | partial               |
| LLM-driven MAS (Simulation) [16]      | ✓                   | ✓           | ✗                | ✗                     |
| LLM–Home Assistant [17]               | ✓                   | ✗           | ✗                | ✓                     |
| <b>SmartHouseOperator (this work)</b> | ✓                   | ✓           | ✓                | ✓                     |

Note: The symbol ✓ denotes explicit support or implementation of the feature, whereas ✗ indicates absence of that capability. “Partial” refers to limited or non-fully deployed real-world implementation.

## 4. SmartHouseOperator Architecture

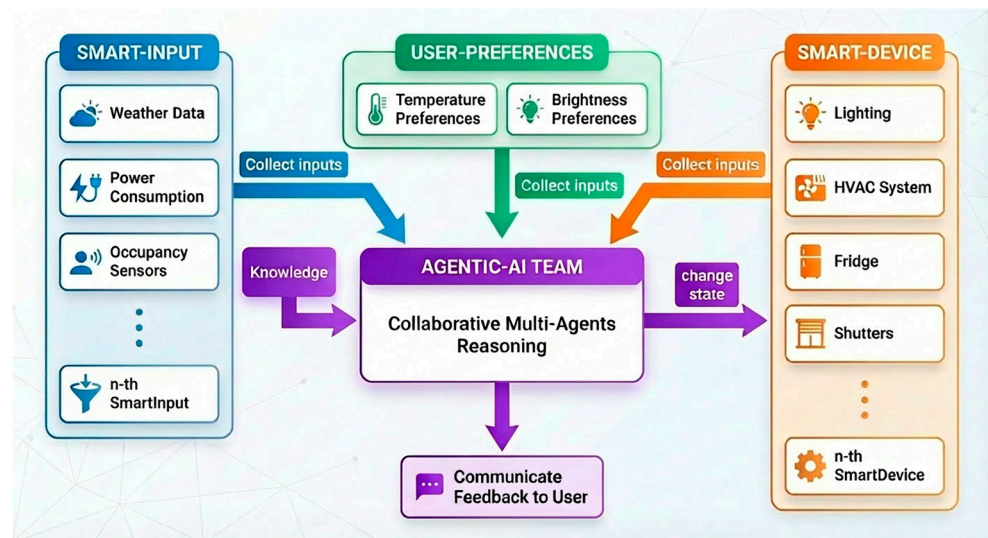
In this section, we introduce the SmartHouseOperator architecture and present a detailed analysis of its system design, core components, and operational structure.

### 4.1. System Architecture

The SmartHouseOperator is built as a modular, agentic-AI framework that integrates configuration metadata, contextual sensing, RL-based preference modeling, device-specific agents, and a central orchestrator to enable adaptive energy-efficient control in smart-home environments. The system is designed to support heterogeneous devices, multiple agentic frameworks, and scalable knowledge integration. The full implementation details are available at *smart-house-operator* GitHub repository [23].

Figure 1 presents a high-level overview of the SmartHouseOperator architecture and its work flow. Contextual information is collected through the SmartInput module, which aggregates dynamic environmental and system signals such as weather conditions, power consumption levels, and occupancy status. In parallel, User-Preferences—including temperature and lighting comfort, are inferred and updated via RL-based preference modeling. These inputs jointly form the perceptual and personalization foundation upon which decisions are made.

At the core of the architecture, the Agentic-AI Team performs collaborative multi-agent reasoning, where each smart device is represented by a dedicated agent that reasons over contextual inputs, learned preferences, and device-specific knowledge. The agents coordinate their decisions to produce a consistent and energy-aware device-state plan, which is then applied to the SmartDevices (e.g., lighting, shutters, fridge or heating, ventilation, and air conditioning (HVAC)) through a unified control interface. The resulting actions close the perception–decision–actuation loop, while the system simultaneously generates explanatory feedback for the user, ensuring transparency and supporting informed human interaction.



**Figure 1.** SmartHouseOperator architecture and five-step perception–reasoning–actuation control cycle.

#### 4.2. Role of the LLM in Agent Collaboration and Decision-Making

Within SmartHouseOperator, LLM is used as high-level reasoning components rather than direct controllers of device actuation. Each Device Agent invokes the LLM to interpret the unified system state, persistent knowledge, and learned preference distributions, and to propose device-specific state adjustments together with a short justification. Collaboration is mediated by a Manager Agent, which consolidates the proposed actions from Device Agents, resolves conflicts against global constraints (e.g., energy budget and comfort preferences), and produces a consistent device-state plan. The selected agentic framework (LangGraph, Agno, or CrewAI) provides the orchestration layer for executing the multi-agent interaction and passing structured messages between agents, while the Operator applies the final plan through the SmartDevice interface.

Within the Agentic AI Team module, the LLM serves as a shared high-level reasoning engine that enables device agents to interpret context, user preferences, and embedded knowledge in a consistent semantic space. Each Device Agent queries the LLM with a structured representation of the unified system state and its device-specific constraints and receives a proposed device action together with a short rationale. Agent collaboration is achieved through structured message passing, where individual proposals are exchanged and evaluated within the agentic orchestration framework. A Manager Agent coordinates this process by aggregating device-level proposals, resolving conflicts against global constraints (e.g., energy budget and comfort priorities), and producing a coherent system-wide action plan. In this way, the LLM supports collaborative reasoning and coordination across agents, while final decision execution remains governed by explicit orchestration logic and device-level validation.

#### 4.3. Core Components

This subsection describes the core components of the SmartHouseOperator architecture and outlines their individual roles within the overall system:

1. Configuration Specs: define the static metadata required for system operation including:
  - household and device specifications,
  - agent descriptions and behavioral guidelines,
  - LLM parameters.

This component provides the declarative foundation for system behavior.

2. **Knowledge Module:** this module supplies the persistent knowledge base for all agents. It stores general guidelines, device-specific rules, environmental constraints, and reasoning principles that inform decision-making and ensure consistent long-term behavior.
3. **SmartInput:** represents a unified interface for collecting contextual environmental signals that influence agent decisions. Typical inputs include:
  - real-time and forecasted weather data,
  - current power consumption and supply constraints,
  - occupancy status and movement patterns,
  - temporal context (time of day, day of week).

These inputs form the perceptual grounding for adaptive reasoning.

4. **SmartDevice:** abstracts all supported controllable devices (e.g., air-conditioning (AC) units, shutters, lights, refrigerators). Each device exposes a unified interface for retrieving its current state and applying state-changing actions.
5. **PreferenceRLAgent (RL Module):** The PreferenceRLAgent uses offline reinforcement learning to infer probabilistic user preference distributions—such as preferred temperature ranges or lighting levels. Using user-provided specifications, it produces preference probabilities distribution that guide downstream reasoning and introduce personalization into the decision-making pipeline.
6. **Agentic AI Framework:** supports interchangeable agentic frameworks, enabling Device Agents to be represented as:
  - graph nodes in LangGraph [24],
  - modular multi-agents managed by a coordinator in CrewAI [25],
  - collaborative team members in Agno [26].

These frameworks coordinate reasoning tasks, plan device adjustments, and interpret system inputs via LLM-powered logic.

7. **Device Agents:** Each SmartDevice is linked to a dedicated Device Agent responsible for:
  - device-specific reasoning,
  - interpreting device-contextual inputs,
  - proposing desired device states to the agentic framework.

Device Agents leverage both general knowledge (e.g., energy-saving principles) and device-specific knowledge (e.g., AC seasons rules, fridge temperature thresholds).

8. **Operator:** serves as the top-level controller governing the system's entire execution cycle. It coordinates sensing, reasoning, learning, and actuation across all components, ensuring consistent behavior and safe execution.

#### 4.4. Operational Flow

This subsection describes the operational flow of the SmartHouseOperator, outlining the structured control cycle through which contextual information, learned user preferences, and device states are continuously integrated to produce coordinated and energy-aware decisions. The five-step control cycle described below defines the complete perception–reasoning–actuation loop executed during each control iteration and is illustrated in Figure 1 to provide a graphical overview of the process. The SmartHouseOperator executes a structured 5-step control cycle:

1. **Data Collection and State Aggregation:** The Operator first gathers all available inputs from:



- SmartInput (current day and time, weather, occupancy, power usage),
- SmartDevice (current device states),
- PreferenceRLAgent (learned preference distributions).

These sources are merged into a unified environment-state object, representing a complete snapshot of the current context.

2. Construction of a Unified Input Representation: The Operator converts the collected information into a structured dictionary that serves as the single input payload for agentic decision-making. This representation ensures consistency across different agent frameworks and device agents.
3. Decision-Making via Agentic AI Framework: The chosen agentic framework (Lang-Graph, Agno, or CrewAI) processes the unified state input by invoking the collaborative team of Device Agents. Each agent evaluates:
  - its own device's state,
  - relevant entries from persistent knowledge and memory,
  - RL-inferred user preferences,
  - global and device-specific guidelines,

and proposes the desired next state for its device. The Manager Agent aggregates these proposals, resolves conflicts using global constraints and learned preferences, and outputs the final device-state plan for execution.

4. Structured Output Generation: The agentic framework returns a strictly formatted output object containing:
  - the desired state for each device,
  - natural-language explanations of the reasoning process,
  - alerts (e.g., over-power conditions, abnormal behavior).

This structured response enables transparent interpretation and reliable execution.

5. Device Actuation: Finally, the Operator updates each device using the unified SmartDevice interface. The system then:
  - applies changes to all devices,
  - sends explanations and alerts (if needed) to the user,
  - closes the loop for the next control cycle.

Through this closed-loop execution cycle, the SmartHouseOperator enables continuous adaptation to dynamic environmental conditions and user behavior, ensuring consistent, transparent, and personalized smart-home control across heterogeneous devices.

In a real smart-home deployment, the proposed framework operates as an orchestration layer on top of existing commercial devices and platforms. Physical devices such as air conditioners, lighting, shutters, and refrigerators are integrated through the SmartDevice abstraction, which maps vendor-specific APIs to a unified control interface. Contextual inputs (e.g., weather, time, occupancy, and power load) are obtained from external services or local sensors, while user preferences are learned incrementally during normal household operation. The Operator executes periodic control cycles, invoking the agentic reasoning layer to produce coordinated device adjustments in real time. The framework has already been deployed and tested in a real residential smart-home, using commercially available devices, and the reported results include data collected from this real-home environment.

## 5. Experiment Setup

The scientific methodology adopted in this study follows a simulation-driven evaluation framework complemented by selected real-device interactions. The simulation model was designed to reflect typical residential smart-home conditions, including

heterogeneous devices, variable environmental context, and evolving user preferences. Model parameters—such as device power baselines, environmental inputs, and preference-learning rates, were selected based on device specifications, commonly reported household energy ranges, and pilot experiments, and are detailed in the corresponding subsections. Boundary conditions, including device operational limits and comfort constraints, are enforced at the agent level to ensure realistic behavior. Experimental results are analyzed using aggregated qualitative and quantitative performance metrics over multiple runs and environmental configurations, as described in the evaluation section.

Experiments were conducted in simulated and also real smart-home environments designed to evaluate the adaptability, personalization capability, and coordination efficiency of the SmartHouseOperator framework. The evaluation setup integrates a locally hosted LLM backend, RL-driven preference modeling, contextual sensing inputs, controllable smart devices, and multiple interchangeable agent-orchestration engines.

Our system's reasoning layer runs on Ollama [27], a local model server deployed on a dedicated machine equipped with 88 CPU (central processing unit) cores, 1 GPU (graphics processing unit), and 128 GB RAM (random-access memory). Running LLMs locally offers several advantages required for experimentation:

- deterministic performance with configurable model versions,
- reduced inference latency,
- full data privacy during environment simulations,
- ability to experiment with multiple locally run LLMs,
- and repeatable experimental conditions without reliance on external APIs.

Although the framework is compatible with any language model supported by Ollama, all experiments in this study were conducted using the gpt-oss:20b model [28], configured with a temperature of 0.2 and a fixed random seed to ensure deterministic and reproducible outputs. This configuration was chosen to balance effective reasoning and instruction-following capabilities with stable, low-variance, and efficient response generation.

Regarding personalization, we employ reinforcement learning (RL)-based persona modeling, where each simulation instance randomly selects a subset of personas assumed to be present in the home. Two independent RL environments are defined: a Temperature Preference Environment and a Brightness Preference Environment. Each environment is learned using a PreferenceRLAgent, implemented as a tabular, model-free Q-learning agent operating in a contextual bandit setting [29].

Formally, each PreferenceRLAgent learns an action–value function  $Q(s, a)$  over discretized contextual states  $s \in \mathcal{S}$  and preference actions  $a \in \mathcal{A}$ , where states encode persona and comfort context, and actions correspond to candidate temperature or brightness settings. Since each interaction consists of a single decision followed by immediate feedback, the learning problem is formulated as a contextual bandit and no discounting of future rewards is applied. The Q-values are updated using the standard tabular Q-learning rule:

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha[r(s, a) - Q_t(s, a)],$$

where  $\alpha$  denotes the learning rate and  $r(s, a)$  represents immediate user-comfort feedback. Action selection follows an  $\epsilon$ -greedy policy to balance exploration and exploitation. After training, the learned Q-values are interpreted as empirical preference strengths and normalized to form probabilistic preference distributions over actions, which are provided as inputs to the agentic reasoning layer to guide personalized device-level decisions.

Although the underlying physical environment is continuous, the agent state is explicitly discretized into a small set of context–action pair representing relevant preference

conditions (e.g., comfort vs. discomfort boundaries). Because each episode consists of a single decision step with immediate feedback, the learning problem reduces to estimating preference probabilities rather than optimizing long-horizon control policies. As a result, no discounting of future returns is applied, and convergence corresponds to stabilization of action preference distributions rather than asymptotic optimal control.

Given the low-dimensional, discretized state representation and bandit-style formulation, short training cycles are sufficient to capture stable preference trends, rather than full policy convergence in a continuous control sense.

Training parameters are fixed across all experiments to ensure consistency and reproducibility. The temperature preference agent is trained for 30 episodes, while the brightness preference agent is trained for 100 episodes, reflecting the wider range of brightness values requiring additional exploration. For both agents, the learning rate is set to 0.1 and the exploration rate to  $\epsilon = 0.2$ . The exploration rate was selected based on preliminary pilot experiments to ensure sufficient exploration during early interactions while maintaining stable preference convergence and is consistent with commonly used  $\epsilon$ -greedy settings reported in the reinforcement learning literature [5]. The agent maintains a Q-table over persona–action pairs and follows an  $\epsilon$ -greedy strategy to balance exploration and exploitation. Rather than converging to a single optimal action, the learned Q-values form probabilistic preference distributions that capture each persona’s comfort boundaries.

For example, learned temperature preferences may exhibit the following patterns:

- Man: Highest probability around 22–24 °C.
- Woman: Preference shifted toward higher temperatures (24–26 °C).
- Dogs: Higher tolerance for lower temperature ranges (20–22 °C).

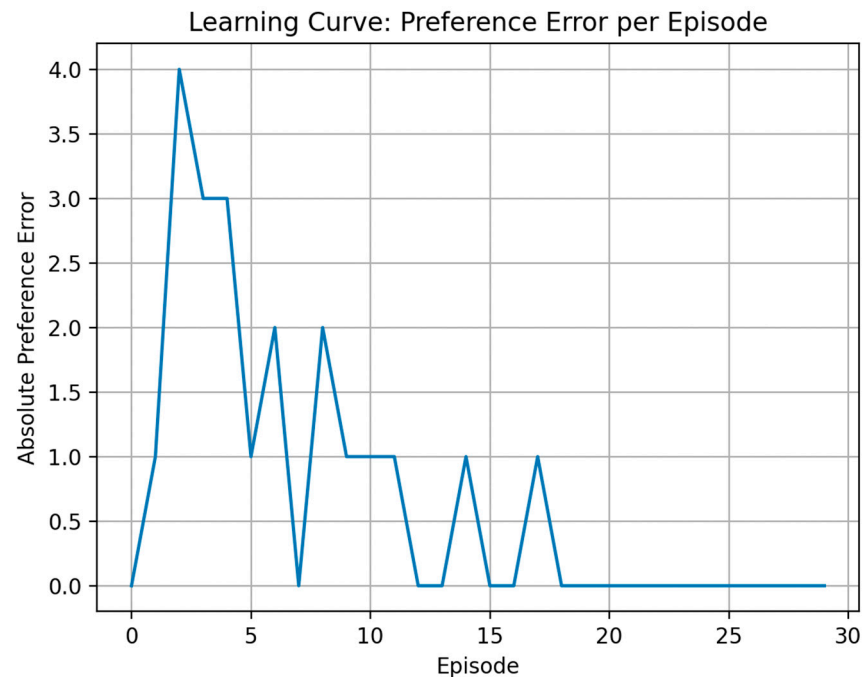
These probabilistic distributions are passed to the agentic reasoning framework and used during decision-making to personalize actions such as AC or lighting control.

To visualize learning dynamics, learning curves are generated during the evaluation phase by tracking the absolute error between the agent’s selected best action and the persona’s ideal preference across training episodes. This representation is well suited to the bandit-style formulation, where reward variance alone may obscure convergence trends (see Figure 2). Here, an episode denotes a single control interaction cycle, consisting of observing the current context, selecting a temperature-setting action, and updating the preference estimation based on the resulting feedback.

Regarding the smart environment inputs, we have 1 real-time and 1 simulated contextual input that serve as perception sources:

- Real-Time Weather Data (temperature, humidity, wind, daylight indicators from Open-Meteo API [30]).
- Simulated Power Consumption categorized into *Normal*, *Moderate*, and *Critical* load levels.

These inputs provide environmental constraints and dynamic conditions under which the agents must operate. In this study, daylight indicators from the Open-Meteo API are represented as a binary day–night flag computed from sunrise to sunset timestamps, without modeling continuous illumination intensity.



**Figure 2.** Learning curve for temperature preference estimation. Each episode corresponds to a single interaction cycle in which the agent observes the environmental state, applies a temperature-setting action, and receives feedback used to update the preference model.

Regarding the controllable smart devices, we incorporated a set of real smart devices, each offering a structured interface for reading and updating the current state:

- Lights—WiZ bulbs [31] (brightness, color warmth, power state);
- AC—Sensibo-controlled air conditioners [32] (temperature, fan speed, mode, power);
- Shutter—Switcher-controlled smart blinds [33] (open/close position);
- Fridge—SmartThings refrigerator [34] (fridge/freezer temperatures).

These devices enable evaluation of multi-agent reasoning under heterogeneous actuation capabilities.

To assess the flexibility of the SmartHouseOperator, experiments evaluate three interchangeable coordination engines:

- CrewAI—structured, role-based teamwork;
- LangGraph—graph-based agent workflows;
- Agno Team—lightweight and modular multi-agents asynchronous coordination.

Each engine is configured to run identical device agents (AC, lighting, shutters, fridge), allowing direct comparison of orchestration behavior, performance, and decision quality.

### 5.1. Main Goals of the Experiment

The primary goal of the experiments is to evaluate the effectiveness of the SmartHouseOperator framework in achieving adaptive, personalized, and coordinated smart-home control using LLM-powered agents and reinforcement-learning-based preference modeling. Specifically, the experiments aim to:

- Assess the quality of multi-agent decision-making across different orchestration engines (CrewAI, LangGraph, Agno), given identical device agents and environmental conditions.

- Evaluate the system's ability to personalize behavior using RL-learned preference distributions and contextual inputs such as weather, power-load level, and occupancy.
- Test system consistency across simulated power-load levels, explicitly examining how decisions change under the Normal, Moderate, and Critical power-consumption scenarios, to determine whether the operator produces reasonable and safe behaviors in each mode.
- Examine the consistency and stability of device-level decisions produced by the operator and agent team across changing environmental states.
- Quantify energy-use patterns of the most power-demanding devices—specifically AC. Due to strong seasonality, occupancy variability, and dynamic changes in home configuration (e.g., new devices added, removal of appliances, travel periods, irregular stays), evaluating total household energy consumption is neither stable nor scientifically comparable over time. Instead, experiments focus on the controllable high-impact devices that dominate energy usage.

Together, these goals allow assessment of both the qualitative intelligence of the system and its quantitative impact on energy-relevant behavior.

### 5.2. Evaluation Metrics

To ensure reliable evaluation in a real home—where total energy use fluctuates with seasons, occupancy, travel patterns, and changes in appliance configuration—the project separates its assessment into two complementary components: (a) the MAS's common-sense reasoning ability, evaluated through multiple simulations with every multi-agents engine, and (b) the system's power-saving efficiency, measured on most power-consuming device (AC) using the best-performing MAS engine configuration.

- MAS's common-sense reasoning evaluation: All qualitative and decision-related metrics are evaluated on a 0–10 scale, where higher scores indicate closer alignment with expected behavior. For each orchestration engine, the final reasoning score is computed as the average over 30 independent simulation runs, covering diverse input conditions. Specifically, each engine is tested 10 times under each power-load level (Normal, Moderate, Critical), with varying simulated occupancy patterns, weather conditions, geographic locations, and times of day. This averaging process ensures that the scoring reflects consistent behavior rather than isolated outcomes. The engines are assessed using the following metrics:

1. **Personalization Accuracy:** How well device decisions match the user's learned preferences.
  - AC: Average deviation (°C) from preferred temperature range.
  - Lighting: Deviation from preferred brightness level.
2. **Contextual Responsiveness:** How consistently the system reacts correctly to changing conditions.

Measured by:

- correct adjustments during weather changes,
  - correct actions under Moderate/Critical/Normal power-load,
  - correct behavior during occupancy patterns.
3. **Interpretability and Traceability:** How clearly the system explains its decisions.
    - clarity of reasoning steps,
    - presence of abnormal-consumption alerts,
    - ability to trace which input (weather, occupancy, preferences) influenced a decision.



4. Response time: How long it took for the system to respond.
- b. The system's power-saving efficiency: The AC power estimation method is based on standard energy performance principles for space heating and cooling, where consumption is proportional to the temperature difference from a reference setpoint and includes additional loads such as fans [35].

To assess the impact of automated control on AC energy efficiency, we extracted input AC settings and the corresponding LangGraph outputs from the results file generated during the previous evaluation of the engines' common-sense reasoning abilities. For each input–output pair, we estimated power consumption using a simplified load model, where total power is calculated as the sum of a base AC load (300 W), a thermal load proportional to the temperature deviation from a reference room temperature ( $\Delta T \times 70 \text{ W/}^\circ\text{C}$ ) multiplied by a mode-specific factor for heating or cooling, and a fan-dependent contribution (30–90 W for low, medium, high). The values for base AC load and fan operations were derived from manufacturer specifications for typical residential devices and cross-checked against reported household power ranges in prior smart-home and energy-management studies [15]. Using this approach, we calculated the percentage improvement in AC power consumption for each scenario across all power consumption levels and aggregated the results to report the estimates for average reductions achieved by the system.

## 6. Experiment Results—Discussion

We begin the evaluation by examining the common-sense reasoning performance of the MAS across the different agentic orchestration engines. Table 2 summarizes the results of this evaluation, reporting averaged scores [0, 10] over multiple simulation runs under diverse environmental conditions and power-load levels. The full evaluation dataset, scoring rubric, and implementation details are available at *smart-house-operator* GitHub repository within the *evaluation* folder [36].

**Table 2.** SmartHouseOperator MAS common-sense reasoning evaluation.

| Metric                                         | CrewAI | Agno  | LangGraph |
|------------------------------------------------|--------|-------|-----------|
| Personalization Accuracy Score (0–10)          | 5.69   | 5.65  | 8.58      |
| Contextual Responsiveness Score (0–10)         | 5.96   | 6.00  | 8.81      |
| Interpretability and Traceability Score (0–10) | 7.35   | 6.88  | 9.69      |
| Response Time (s)                              | 208.36 | 67.33 | 54.26     |

Overall, the results indicate clear performance differences between the evaluated frameworks, with LangGraph consistently outperforming CrewAI and Agno across most metrics. This advantage is primarily attributable to architectural alignment with the task characteristics, rather than inherent superiority of the framework itself. LangGraph provides explicit state management, deterministic execution graphs, and fine-grained control over agent transitions, which closely match the structured, multi-step decision-making and evaluation loops required in this setting. In contrast, CrewAI and Agno emphasize higher-level autonomy and flexible agent behaviors, which introduce additional coordination overhead and less predictable execution paths for tightly controlled workflows. As a result, LangGraph's design is particularly well suited to tasks requiring repeatable reasoning steps, explicit validation stages, and low orchestration latency.

Under critical power-consumption conditions, the SmartHouseOperator demonstrates context-aware and coordinated reasoning across devices. When environmental data indicate daylight and moderate external temperatures, lighting is turned off or dimmed to minimize load, while shutters are partially opened to maximize natural

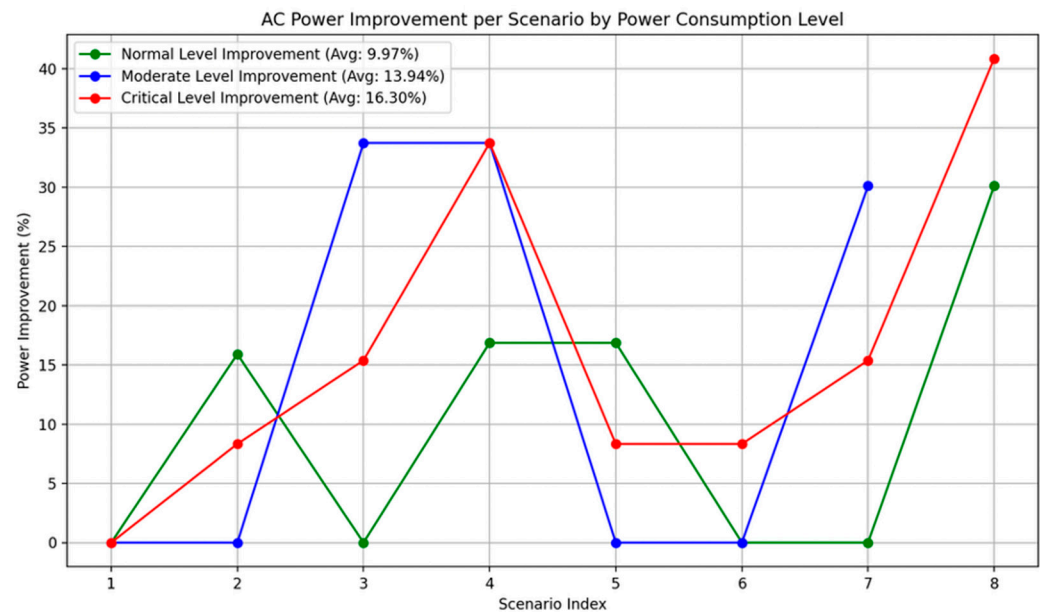
illumination without introducing thermal inefficiencies. The air-conditioning strategy adapts conservatively: in the absence of extreme weather, the AC either remains off or operates at a higher setpoint (e.g., 26 °C with low fan speed) to reduce energy demand, while stricter power constraints trigger further setpoint adjustments. Refrigeration settings follow device-specific rules, maintaining default temperatures under mild conditions and applying slight cooling offsets only when external temperatures exceed defined thresholds. Together, these decisions illustrate how the system balances comfort, environmental context, and energy constraints through coherent multi-device coordination rather than isolated, rule-based actions.

As shown in Table 2, LangGraph achieves the highest performance across all qualitative reasoning metrics, including personalization accuracy, contextual responsiveness, and interpretability. The substantial gap in personalization accuracy suggests that LangGraph is more effective at integrating reinforcement-learning-derived user preferences into coordinated decision-making. Similarly, its superior contextual responsiveness indicates more reliable adaptation to changing environmental and power-load conditions.

In addition to qualitative improvements, LangGraph also exhibits the lowest average response time, highlighting its efficiency in coordinating multi-agent reasoning under identical experimental conditions. In contrast, CrewAI shows comparatively higher interpretability scores but suffers from significantly longer response times, while Agno demonstrates faster execution than CrewAI but lower reasoning quality overall. These results suggest a trade-off between reasoning depth and execution efficiency in some orchestration frameworks. Overall, the findings indicate that LangGraph provides the best balance between reasoning quality, consistency, and performance, making it the most suitable engine for the subsequent energy-efficiency evaluation.

Regarding the evaluation of (b) The system's power-saving efficiency, the detailed methodology used to estimate AC power consumption improvements is available at smart-house-operator GitHub repository within the *evaluation* folder. Figure 3 summarizes the estimated power savings achieved by the SmartHouseOperator across all simulated scenarios in which the air-conditioning system was active, grouped by power-consumption level (Normal, Moderate, and Critical). Scenario indices in Figure 3 correspond to valid AC control instances per operating condition. In some scenarios, the AC is inactive and therefore excluded from improvement calculations, resulting in fewer plotted samples for that condition. The figure is intended to illustrate overall improvement trends and average behavior rather than strict scenario-by-scenario alignment.

As illustrated in Figure 3, the SmartHouseOperator consistently reduces AC power consumption across all operating conditions. Under Normal power-load scenarios, the system achieves an average improvement of approximately 10%, while savings increase to nearly 14% under Moderate load and reach over 16% in Critical power-consumption conditions. These results are aggregated over eight representative simulated scenarios that differ in weather conditions, occupancy patterns, and baseline energy demand. This trend indicates that the system adapts more aggressively as energy constraints become stricter, prioritizing efficiency while maintaining acceptable comfort levels. Overall, the results demonstrate that the automated, agentic control strategy is particularly effective in high-load situations, where coordinated and context-aware adjustments yield the greatest energy benefits. In Scenario 8, the Moderate operating level is not reported because the air-conditioning system is inactive in that scenario; as a result, no AC power improvement can be computed, and the scenario is excluded from the improvement analysis by design.



**Figure 3.** AC power improvements by power consumption levels.

The system's performance exhibits moderate sensitivity to the choice of LLM and prompt configuration, primarily affecting response fluency and reasoning verbosity rather than overall task completion or decision quality. Core system behavior is governed by the agentic control structure and explicit orchestration logic, which remain consistent across different LLM backends. While higher-capacity models generally improve reasoning consistency, the system does not rely on highly specialized prompts or model-specific features. Lightweight, task-oriented prompts were intentionally used to reduce overfitting to specific LLM behaviors and to enable substitution with local or self-hosted models. As a result, performance differences across LLMs are secondary to architectural design choices and task–framework alignment.

## 7. Design Scope and System Limitations

The following limitations are discussed in the context of the proposed framework design and its positioning relative to existing smart-home AI approaches.

This research prototype demonstrates multi-agent reasoning for energy-aware smart-home control but currently operates with limited user interaction and sensory grounding. Explicit human-in-the-loop (HITL) mechanisms are not yet supported, meaning users cannot directly influence agent decisions, provide corrective feedback, or override actions during runtime. In addition, the system relies on simulated household personas, occupancy signals, and power consumption data, which constrains contextual realism. These design choices were made to emphasize architectural clarity and agent coordination rather than full environmental fidelity.

Future iterations can address these limitations by incorporating real-time human feedback, live sensor streams, and richer perception capabilities. Enabling HITL interaction would improve preference alignment, transparency, and trust by allowing users to guide and refine agent behavior dynamically. Integrating visual perception and real-time power monitoring would further enhance contextual awareness and responsiveness.

The proposed framework adopts a multi-agent architecture and assumes reliable communication between agents and the orchestration layer. While communication security, resilience to interruptions, and robustness against adversarial attacks (e.g., false data injection) are critical concerns in general multi-agent control systems, these aspects are outside the scope of the present study, which focuses on architectural integration and

adaptive decision-making at the application level. Existing work has extensively studied secure and fault-tolerant control strategies for multi-agent systems under communication disruptions and adversarial conditions. Integrating such mechanisms into agentic, LLM-driven smart-home systems represents an important direction for future work [37,38].

## 8. Conclusions and Future Directions

This work introduced the SmartHouseOperator, a multi-agent intelligent control framework designed to bring adaptability, coordination, and energy efficiency to modern smart homes. By integrating LLM-powered reasoning, RL-based preference modeling, and contextual sensing, the system demonstrates how autonomous decision-making can overcome the limitations of static rules, fragmented device control, and non-personalized automation found in today's smart-home solutions.

Through a unified evaluation pipeline combining simulated scenarios and real-home inputs, the framework consistently showed robust reasoning across different orchestration engines, with LangGraph achieving the highest performance. Energy-focused experiments further confirmed the system's practical benefits: the SmartHouseOperator achieved notable reductions in AC power consumption, from about 10% under Normal load up to 16% under Critical power conditions, demonstrating its ability to respond intelligently to increasing energy constraints.

Overall, the results highlight the potential of hybrid AI architectures—combining deep learning for perception, LLMs for reasoning, reinforcement learning for adaptive behavior, and persistent memory for contextual awareness—to create smart homes that are not only automated, but genuinely intelligent, personalized, and responsive to real-world dynamics.

While the SmartHouseOperator demonstrates the feasibility of agentic, energy-aware smart-home control, several extensions can further enhance its adaptability and real-world applicability. A key future direction is the incorporation of HITL interaction through selective approval and override mechanisms. In particular, the system may operate in a semi-autonomous mode, where certain actions—such as extreme changes to HVAC settings or refrigerator states, require explicit user approval before execution. This will enable users to intervene when needed, provide real-time corrective feedback, and maintain oversight without disrupting routine autonomous operation. Such an approach enhances preference alignment, transparency, and user trust, while preserving the benefits of agentic automation.

Additional enhancements include the integration of real-time visual perception for accurate occupancy and activity recognition, and live power monitoring via smart meters or IoT sensors to enable finer-grained, responsive energy optimization. More broadly, future work will explore hybrid cognitive architectures that combine deep learning for perception, LLM-based reasoning for coordination and explanation, reinforcement learning for adaptive preference modeling, and persistent memory for contextual awareness. Together, these extensions would move the SmartHouseOperator closer to fully autonomous, trustworthy, and human-aligned intelligent home orchestration.

**Author Contributions:** Conceptualization, E.D.; methodology, E.D. and C.K.; software, E.D.; validation, E.D.; formal analysis, E.D.; investigation, E.D.; resources, E.D.; data curation, E.D.; writing—original draft preparation, E.D. and C.K.; visualization, E.D.; project administration, E.D. and C.K.; writing—review and editing, C.K.; supervision, C.K. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research received no external funding.

**Data Availability Statement:** The datasets, source code, and implementation details supporting the findings of this study are openly available in the GitHub repository SmartHouseOperator at <https://github.com/edolinina/smart-house-operator> (accessed on 08 Dec 2025).

**Conflicts of Interest:** The authors declare no conflict of interest.

## References

1. Lobaccaro, G.; Carlucci, S.; Löfström, E. A Review of Systems and Technologies for Smart Homes and Smart Grids. *Energies* **2016**, *9*, 348. <https://doi.org/10.3390/en9050348>.
2. Rehman, U.U.; Faria, P.; Gomes, L.; Vale, Z. Future of Energy Management Models in Smart Homes: A Systematic Literature Review of Research Trends, Gaps, and Future Directions. *Process Integr. Optim. Sustain.* **2025**, *9*, 1169–1198. <https://doi.org/10.1007/s41660-025-00506-x>.
3. Elmi, Y.A. Interoperable IoT Devices and Systems for Smart Homes: A Data Analytics Approach to Enhance User Experience and Energy Efficiency. *J. Digitainability Realism Mastery (DREAM)* **2023**, *2*, 51–66. <https://doi.org/10.56982/dream.v2i10.195>.
4. Wei, T.; Wang, Y.; Zhu, Q. Deep Reinforcement Learning for Building HVAC Control. In *Proceedings of the 54th Annual Design Automation Conference (DAC)*; IEEE: Austin, TX, USA, 2017.
5. Sutton, R.S.; Barto, A.G. *Reinforcement Learning: An Introduction*; MIT Press: Cambridge, MA, USA, 2018.
6. Wooldridge, M. *An Introduction to MultiAgent Systems*; Wiley: Chichester, UK, 2009.
7. Jennings, N.R.; Sycara, K.; Wooldridge, M. A Roadmap of Agent Research and Development. *Auton. Agents Multi-Agent Syst.* **1998**, *1*, 7–38. <https://doi.org/10.1023/A:1010090405266>.
8. Langer, M.; König, C.; Back, C.; Hemsing, V. Trust in Artificial Intelligence: Comparing Trust Processes Between Human and Automated Trustees in Light of Unfair Bias. *PsyArXiv* **2021**. <https://doi.org/10.31234/osf.io/r9y3t>.
9. Fragiadakis, G.; Diou, C.; Kousiouris, G.; Nikolaidou, M. Evaluating Human–AI Collaboration: A Review and Methodological Framework. *arXiv* **2025**, arXiv:2407.19098.
10. Li, S.; Guo, Y.; Yao, J.; Liu, Z.; Wang, H. HomeBench: Evaluating LLMs in Smart Homes with Valid and Invalid Instructions Across Single and Multiple Devices. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*; Association for Computational Linguistics: Vienna, Austria, 2025; pp. 12230–12250.
11. Chang, Y.; Wang, X.; Wang, J.; Wu, Y.; Yang, L.; Zhu, K.; Chen, H.; Yi, X.; Wang, C.; Wang, Y.; et al. A Survey on Evaluation of Large Language Models. *ACM Trans. Intell. Syst. Technol.* **2024**, *15*, 39. <https://doi.org/10.1145/3641289>.
12. Cui, H.; Du, Y.; Yang, Q.; Shao, Y.; Liew, S.C. LLMind: Orchestrating AI and IoT with LLM for Complex Task Execution. *IEEE Commun. Mag.* **2025**, *63*, 214–220. <https://doi.org/10.1109/MCOM.002.2400106>.
13. Rivkin, D.; Hogan, F.; Feriani, A.; Konar, A.; Sigal, A.; Liu, S.; Dudek, G. SAGE: Smart Home Agent with Grounded Execution. *arXiv* **2024**, arXiv:2311.00772.
14. Latorń, D.; Grela, J.; Ożadowicz, A. Applications of Deep Reinforcement Learning for Home Energy Management Systems: A Review. *Energies* **2024**, *17*, 6420. <https://doi.org/10.3390/en17246420>.
15. Jimenez-Romero, C.; Yegenoglu, A.; Blum, C. Multi-Agent Systems Powered by Large Language Models: Applications in Swarm Intelligence. *Front. Artif. Intell.* **2025**, *8*, 1593017. <https://doi.org/10.3389/frai.2025.1593017>.
16. Giudici, M.; Sironi, A.; Villa, I.; Scherini, S.; Garzotto, F. Generating HomeAssistant Automations Using an LLM-Based Chatbot. *arXiv* **2025**, arXiv:2505.02802.
17. Home Assistant. Available online: <https://www.home-assistant.io> (accessed on 18 December 2025).
18. Amershi, S.; Weld, D.; Vorvoreanu, M.; Fournery, A.; Nushi, B.; Collisson, P.; Suh, J.; Iqbal, S.; Bennett, P.N.; Inkpen, K.; et al. Guidelines for Human–AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19)*, Glasgow, UK, 4–9 May 2019; pp. 1–13. <https://doi.org/10.1145/3290605.3300233>.
19. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-T.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv* **2020**, arXiv:2005.11401.
20. Wang, Z.; Mu, C.; Hu, S.; Chu, C.; Li, X. Modelling the Dynamics of Regret Minimization in Large Agent Populations: A Master Equation Approach. In *Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI-22)*, Vienna, Austria, 23–29 July 2022. Available online: <https://www.ijcai.org/proceedings/2022/0076.pdf> (accessed on 21 January 2026).
21. Wang, X.; Guang, W.; Huang, T.; Kurths, J. Optimized Adaptive Finite-Time Consensus Control for Stochastic Nonlinear Multiagent Systems with Non-Affine Nonlinear Faults. *IEEE Trans. Autom. Sci. Eng.* **2024**, *21*, 5012–5023. <https://doi.org/10.1109/TASE.2023.3306101>.



22. Wang, X.; Pang, N.; Xu, Y.; Huang, T.; Kurths, J. On State-Constrained Containment Control for Nonlinear Multiagent Systems Using Event-Triggered Input. *IEEE Trans. Syst. Man Cybern. Syst.* **2024**, *54*, 2530–2538. <https://doi.org/10.1109/TSMC.2023.3345365>.
23. Dolinin, E. Smart-House-Operator. Available online: <https://github.com/edolinina/smart-house-operator> (accessed on 18 December 2025).
24. LangChain. LangGraph. Available online: <https://www.langchain.com/langgraph> (accessed on 18 December 2025).
25. CrewAI. CrewAI. Available online: <https://www.crewai.com> (accessed on 18 December 2025).
26. Agno. Agno Agent Framework. Available online: <https://www.agno.com/agent-framework> (accessed on 18 December 2025).
27. Ollama. Available online: <https://ollama.com> (accessed on 18 December 2025).
28. OpenAI. Introducing GPT-OSS. Available online: <https://openai.com/index/introducing-gpt-oss> (accessed on 18 December 2025).
29. Langford, J.; Zhang, T. The Epoch-Greedy Algorithm for Contextual Multi-Armed Bandits. *Adv. Neural Inf. Process. Syst.* **2008**, *20*, 817–824.
30. Open-Meteo. Available online: <https://open-meteo.com> (accessed on 18 December 2025).
31. Wiz Connected. Wiz Smart Bulbs. Available online: <https://www.wizconnected.com/en-us/products/bulbs> (accessed on 18 December 2025).
32. Sensibo. Sensibo Smart AC Controllers. Available online: <https://sensibo.com> (accessed on 18 December 2025).
33. Switcher. Switcher Smart Home Products. Available online: <https://switcher.co.il> (accessed on 18 December 2025).
34. Samsung. SmartThings. Available online: <https://www.samsung.com/us/smartthings> (accessed on 18 December 2025).
35. EN ISO 13790; Energy Performance of Buildings—Calculation of Energy Use for Space Heating and Cooling. European Committee for Standardization (CEN); Brussels, Belgium, 2008.
36. Dolinin, E. Smart-House-Operator Evaluation. Available online <https://github.com/edolinina/smart-house-operator/tree/main/evaluation>(accessed on 18 December 2025).
37. Heemels, W.P.M.H.; Donkers, M.C.F.; Teel, A.R. Periodic Event-Triggered Control for Linear Systems. *IEEE Trans. Autom. Control* **2013**, *58*, 847–861. <https://doi.org/10.1109/TAC.2012.2220443>.
38. Pasqualetti, F.; Dörfler, F.; Bullo, F. Control-Theoretic Methods for Cyberphysical Security: Geometric Principles for Optimal Cross-Layer Resilient Control Systems. *IEEE Control Syst. Mag.* **2015**, *35*, 110–127. <https://doi.org/10.1109/MCS.2014.2364725>.

**Disclaimer/Publisher’s Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.